

Hackathon Information Sheet

DS6051 – Decoding Large Language Models

Format	In-group, hands-on build session with a final presentation
Group Size	3–5 students per group
Submission	Devpost — project overview, github repo, results table, and ~5-minute video
Peer Feedback Due	EOD July 10

Overview

Over a single six-hour session, your group will build a safety evaluation scorecard for an LLM. Each group selects the domain and the metrics they want to include in their scorecard, then implements and runs those metrics against a provided model. For the purposes of this hackathon, you will be acting as if the general-purpose LLM we provide you was created for the purpose of your use-case.

Groups will create a complete pitch for a safety scorecard — all the metrics that should be included and why, specific to your chosen domain and use-case — and implement them during the hackathon. The main goal is to grapple with three questions: **What should we measure, Why should we measure it, and How can we measure it?** Approach each from a model architecture standpoint and note the limitations inherent in every measurement.

This is an evaluation exercise, not a training exercise. You are not improving, fine-tuning, or otherwise training the model in any way. The work is figuring out how to correctly evaluate the necessary safety dimensions for your use-case and recognize the limitations of those measurements given by the architecture of current-day LLMs.

Learning Goals

- Translate an abstract safety concern (“is this model safe enough to deploy as a medical triage assistant?”) into concrete, measurable metrics.
- Build and run evaluation code against a real open-weight model, including an LLM-as-judge pipeline where appropriate.
- Reason critically about what a metric does and does not capture, and where the architecture of the model limits what you can actually measure.
- Communicate findings clearly: what the numbers mean for real-world deployment in your domain.

Suggested Day-Of Flow

You are free to manage your own time, but a rough cadence that works well for a six-hour build:

1. Draft your scorecard pitch: the metrics you intend to include and the justification for each, specific to your use-case - Divide work across the group
2. Stand up the inference pipeline and confirm that you can get coherent outputs from both models.

3. Assemble or source evaluation dataset(s) specific to your metrics and domain.
4. Build and test each metric; wire up the LLM-as-judge pipeline for any open-ended scoring. Also, add two more models as LLM-as-judge, and compare their relative scores.
5. Run the full scorecard, record results, and interpret what they mean for your use-case.
6. Write your project overview, compile your results table, write your discussion, and record your ~5-minute video, then submit on Devpost.

Final Product

Each group submits the following:

- **Full scorecard** — the complete set of metrics with written justifications for why each belongs to your scorecard for this domain/use-case.
- **Code on GitHub** — a repository that creates and tests each metric on the LLM, structured so others can follow what you did.
- **Discussion of how metrics were evaluated** — the evaluation dataset(s) specific to your metrics/domain where needed, and how the LLM-as-judge was used, including its limitations.
- **A table** of your scorecard results
- **Discussion of your results** — how did the LLM do on each metric, and what does that mean for your domain/use-case? Be explicit about what the measurement can and cannot tell you.
- **Devpost submission**: upload a project overview, github repo, results table, written discussion, and ~5-minute video

Potential Metrics to Include

A **non-exhaustive** menu of safety dimensions. Choose metrics that matter most for your specific use-case and be ready to justify why each belongs.

Metric	Result	Why it Needs to be Included for this Use Case	How it's Measured (automated, LLM-as-judge, or human-required)	Limitations
Ex. Attack Success Rate	0.72		Automated (using X dataset)	

Truthfulness & grounding

- Faithfulness / hallucination rate
- Citation accuracy / source attribution — does it cite real, relevant sources?
- Factual consistency — does it contradict itself across a conversation?
- Calibration — does stated confidence match actual accuracy?

Robustness & security

- Robustness to rephrasing/noise
- Attack success rate / jailbreak resistance

Fairness & bias

- Bias (many types of this) — pick *specific* axes (ex. demographic, dialect/language/culture, political, etc.)

- Performance disparity / fairness gap
- Stereotype reinforcement rate

Privacy & data

- PII leakage rate
- Training-data memorization / regurgitation — will it reproduce verbatim text it shouldn't?
- Context-confidentiality — does it leak info from one user/session into another?

Behavioral / alignment

- Refusal behavior — two-sided metric: over-refusal *and* under-refusal
- Sycophancy
- Toxicity
- Instruction-following fidelity — does it actually do what's asked, including honoring constraints/format?
- Steerability — can it be reliably constrained by a system prompt? (e.g., "never give medical dosages")

Reasoning quality

- Explainability
- Plausibility
- Reasoning faithfulness — does the stated chain-of-thought actually reflect how it got the answer?

Practical / deployment

- Latency & cost — how long does it take to check the safety of every prompt? Is this feasible?
- Failure-mode graceful degradation — what does it do when it doesn't know? (abstention rate)
- Coverage / answer rate — what fraction of in-domain queries does it actually attempt?

Cross-Lingual Safety Distribution Shift

While modern foundation models exhibit robust safety guardrails when interacting in English, their alignment frequently degrades when faced with non-English inputs. This phenomenon—known as a cross-lingual safety distribution shift—creates critical vulnerabilities where a model safely refuses a harmful or hazardous prompt in English yet complies with the exact same query when translated into another language. **To evaluate this for your scorecard, select 1-2 additional languages alongside English (we highly recommend testing at least one low-resource language). Translate a subset of your domain-specific evaluation dataset into these chosen languages and measure the safety gap in the model's refusal rates or accuracy compared to the English baseline.**

Bonus points: propose an entirely new safety-evaluation metric or capability for your domain. For example, researchers have tried to evaluate the deception / sandbagging capability of LLMs in high-stakes applications, where a model can change or hide its reasoning.